

FINAL YEAR PROJECT · PROJECT REVIEW

Real-Time ECG Heart Visualizer

We built a single-lead electrocardiograph whose main output is not a chart but an anatomically accurate 3-D human heart. It contracts on every detected heartbeat, in time with synthesised heart sounds, above the live waveform the beat was measured from.



● SYSTEM LIVE

1000 HZ

AD8232 · ARDUINO UNO R3

● AWAITING HARDWARE

Deployed at <https://143-198-27-18.nip.io>

Why we put the anatomy in front of the trace

An electrocardiogram records the electrical activity that drives each heartbeat. Its familiar shape carries three events: the **P wave** as the atria contract, the **QRS complex** as the ventricles contract, and the **T wave** as the ventricles recover.

A conventional monitor draws that shape and stops there. Reading it well depends on already holding a picture of the organ it came from, and that picture is the one thing the trace itself does not supply.

So we inverted the output. The waveform stays, because it is the measurement and we did not want to hide it. But we drove an anatomical heart from the same detected beats and gave it the centre of the screen.

Clinically honest

We implemented the same filtering and detection a hospital monitor uses, rather than a simple peak finder.

Anatomically real

Four chambers, great vessels and coronary arteries. Not a symbol.

Physiologically timed

We schedule the atria to contract before the ventricles, which is the order the real organ works in.

Reachable

It runs in a browser on any machine, with or without a network connection.

The system as it runs today



One decision that shaped everything after it

We settled early on a single contract: every data source emits **raw 10-bit ADC counts at 1000 Hz**, exactly what `analogRead(A0)` returns on the Arduino. Because we held to it, nothing downstream ever learns which source is connected.

SOURCE

Simulated AD8232, real board over USB, or samples forwarded from a browser

FILTER

0.5 Hz high-pass → 50/100 Hz notch → 40 Hz low-pass, stateful across chunks

DETECT

Streaming Pan-Tompkins with an adaptive threshold

CHANNEL

Per-client bounded queue, so a slow viewer never stalls acquisition

RENDER

3-D heart, waveform, heart sounds, CSV recorder

What it bought us

Swapping the simulation for real hardware became a configuration change instead of a rewrite. By the time the board arrives, everything downstream will already have run for hours.

Three implementations

We wrote `SimulatedSource`, `SerialSource` and `ClientFedSource` against the same contract. Each is a peer; we resisted making any of them a special case.

Backpressure

We give each viewer its own queue, so a struggling browser drops only its own frames and sampling stays at a true 1000 Hz. **Display may degrade; timing must not.**

Finding the beat in a noisy millivolt

At the skin the heartbeat is about one millivolt, and it arrives buried in interference: mains hum at 50 Hz, electrical noise from other muscles, and a slow drift as the chest rises and falls.

We started with a fixed voltage threshold and abandoned it. It is defeated three ways: the baseline wanders past it as the patient breathes, a tall T wave can cross it, and the signal collapses below it when an electrode dries out.

Pan-Tompkins avoids all three by looking for QRS *energy* in the 5–15 Hz band and adapting its threshold to the running signal and noise levels. It is what real monitors use, so we implemented it in streaming form.

BAND-PASS

5–15 Hz

DERIVATIVE

5-point

SQUARE

rectify

INTEGRATE

150 ms

THRESHOLD

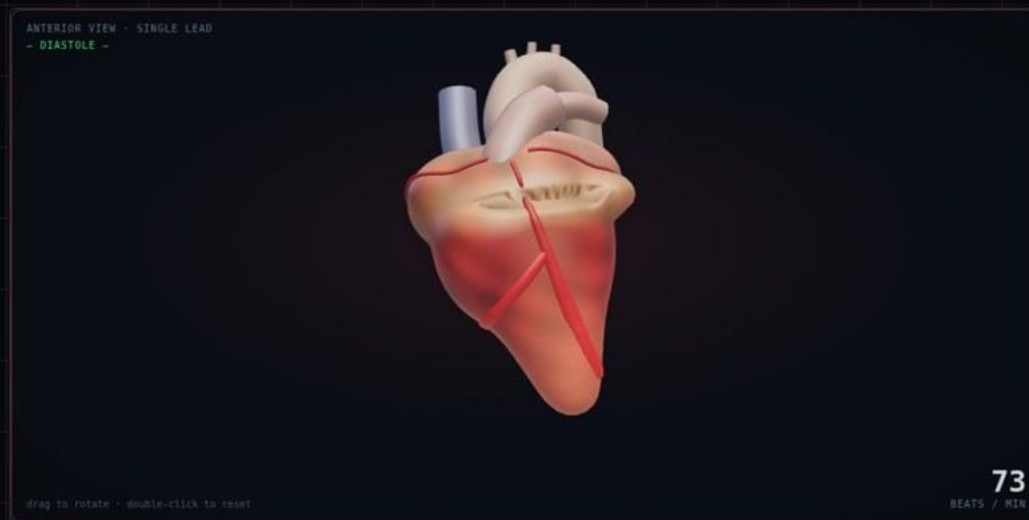
adaptive

Then we had to correct for lag. The integrator peaks roughly 100 ms after the true beat, so we refine each detection against the filtered signal. Before we added that step, the animation and the sound both fired visibly late.



Filtered trace (green) over the raw signal, with detected R-peaks marked.

We generated the heart rather than downloading one



Built at startup from a signed distance field. 16k triangles, one draw call.

A signed distance field describes a solid by answering one question at any point in space: how far is the nearest surface, and am I inside or outside it. We wrote the heart as such a field and walk a sphere of points onto it to produce the mesh.

It carries both ventricles, both atria, the auricles, the coronary sulcus and interventricular groove, the aortic arch with its three head vessels, the pulmonary trunk, both venae cavae, and the LAD, circumflex and RCA arteries resampled onto the muscle surface.

- ✓ Nothing to download or license, so it works offline
- ✓ One continuous surface instead of separate parts pushed together
- ✓ Because the field is analytic, we bake shading and contraction weights straight into the mesh
- ✓ The whole organ draws in **one call**, and a beat costs four numbers

Orientation cost us a correction. +x is the viewer's right and the *patient's left*, which puts the apex down to the viewer's right. Our first version had it mirrored.

Animating a cycle rather than a pulse

Ventricular systole

We fire this on the detected beat and combine three motions: a radial squeeze, a shortening along the long axis, and a counter-rotation between apex and base. That last one is the wringing motion a real ventricle makes, and adding it was what stopped the model looking like a squeezing ball.

Atrial systole

The atria contract about 160 ms *before* the ventricles, so we cannot wait for the beat to arrive. We predict the next one from the running interval and schedule the atrial squeeze ahead of it.

Ejection fraction

Ejection fraction is the share of blood a ventricle expels per beat, around 62% in a healthy adult, and we model the volume accordingly. We also scale contraction with $\sqrt{RR}$, so as the rate climbs it is the filling phase that shortens, which is what happens in life.

S1 and S2

The two sounds are valves closing. “Lub” is the mitral and tricuspid valves at the start of the beat; “dub” is the aortic and pulmonic valves at the *end* of ejection. We space them by the contraction interval rather than a fixed gap, because that is what sets them apart in the real organ.

We ordered the code deliberately. A beat reaches the heart and the audio engine before any page updates run. We found that letting layout work sit between the detection and the animation was enough to make the beat feel late.

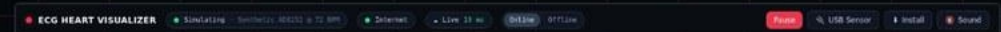
Putting it somewhere it stays running

We host the system on an Ubuntu server, run under `systemd` so it restarts on any exit and survives a reboot. Everything answers on one address.

CONCERN	RESOLUTION
Uptime	<code>systemd</code> , <code>Restart=always</code> , survives reboot
TLS	Caddy + Let's Encrypt, auto-renewing
One link	bare IP and port 80 both 308-redirect to it
Mains hum	notch set to 50 Hz for Nigeria

We needed HTTPS, not for appearance. Browsers refuse to open a USB port or install an application unless the page came over a secure connection, so without a certificate two of our main features simply cannot run.

This address is temporary. We are currently served from the server's own IP with a certificate issued against it, which is enough for development and for this review. When we move to production we will register a domain and host the site under it. Only the address changes; the application does not.



The status bar: source, connectivity, measured latency, and the processing-mode switch.

20 ms

ROUND TRIP

0

DROPPED FRAMES

Three ways to run it, from one codebase

Shared (online)

Everyone holding the link shares one session. When somebody moves a slider, every other dashboard follows, controls included.

For demonstration and teaching.

Private (own sensor)

Once a viewer asks for their own sensor we take them out of the shared session. Nobody else can alter what they see, and they alter nothing for anyone.

Because your trace is your heart.

Offline (installed)

Once installed it runs entirely on the local machine. No server, no network, and the sensor is read directly.

For use where there is no connection.

We made the online mode fall back to local processing the moment the connection drops, and return by itself when it comes back, so **the trace never stops**. Where somebody has deliberately chosen offline, we leave them there.

What using it actually involves

Both routes below are the whole of what a first-time visitor does. We worked to keep them short enough that neither needs explaining.

PATH A · SIMULATION, NO HARDWARE

- 1 Open the link** Nothing to install. The dashboard loads and joins the shared session.
- 2 Press Start Monitoring** The simulated AD8232 begins streaming at 1000 Hz.
- 3 Set a heart rate** Anything from 30 to 200 BPM. Add noise and motion artifacts to watch the filters work.
- 4 Watch and listen** The heart contracts on each R-peak. Press Sound for the lub-dub.
- 5 Record, then export CSV** Raw counts, filtered millivolts and flagged R-peaks, at full 1000 Hz resolution.

PATH B · WITH A REAL ECG SENSOR

- 1 Open the secure link** Required. A browser will not open a USB port over an insecure connection.
- 2 Install the app (optional)** Unlocks offline mode, so the reading never depends on the network.
- 3 Wire the AD8232** Three electrodes on the chest, board to the computer by USB.
- 4 Press USB Sensor** The browser asks which board to use. Granted once, then remembered.
- 5 Press Arduino** The dashboard leaves the shared session. The trace on screen is now yours alone.
- 6 Record, then export CSV** Identical format to Path A, so the two are directly comparable.

Where no board is found, we stop the trace. We chose not to fall back to the simulation, because a moving trace on a dashboard somebody has just wired to their own chest would be read as their heartbeat.

Making it work offline turned out to be the hard part

We assumed offline would be a caching problem and it was not. A service worker caches HTML and JavaScript, but it cannot cache a running Python process, and our simulator, filters and detector were all Python. The signal chain had to exist **in the browser as well**.

We considered rewriting the filter design in JavaScript and decided against it. Filter design is delicate numerical work, and its failure mode is the worst kind: a trace that looks perfectly normal while reporting a different heart rate. Instead we export the SciPy coefficients when we build, and ported only the twenty lines of arithmetic that apply them.

Then we used the Python as an **oracle**, feeding both chains identical samples and comparing what comes out.

Identical raw ADC input, both chains

BPM	noise	filtered	trace	beats
60	0.00	max	diff 0.0e+00	10 = 10
60	0.30	max	diff 0.0e+00	10 = 10
120	0.60	max	diff 0.0e+00	21 = 21
150	1.00	max	diff 0.0e+00	21 = 21
45	0.15	max	diff 0.0e+00	8 = 8

R-peak index drift 0 samples

Reported BPM difference 0.000

Bit-exact, not merely close.

This is why we can say offline and online will not quietly disagree about a heart rate. That disagreement was the one failure that would have made the whole system untrustworthy.

What we have actually measured

20/20

SIGNAL-CHAIN CASES

86/87

BROWSER CHECKS

20/20

MULTI-CLIENT CHECKS

 ± 0.5

BPM ACCURACY

Signal-chain regression (the oracle)

BPM	noise	beats	measured	err	xRT
60	0.00	18	60.61	0.61	56x
60	1.00	18	59.94	0.06	55x
120	1.00	36	120.48	0.48	52x
150	1.00	45	149.63	0.37	50x
180	0.60	55	180.72	0.72	49x

All 20 cases passed.

xRT = times faster than real time

We test from 45 to 180 BPM, from a clean signal to fully noisy, and through step changes that deliberately invalidate the thresholds the detector has already learned. Accuracy holds throughout.

The chain runs roughly **50x faster than real time**, which tells us the backend sits near idle while streaming rather than keeping pace with effort.

Our browser suite drives the real application through a headless Chrome. It includes **cutting the network and reloading**, after which the app still starts and still finds beats.

Four corrections, and what led to each

A slow viewer was stalling the sampler

We were sending each batch straight to the socket. A browser that fell behind then applied backpressure all the way to the sampling loop, and we watched the display drift *seconds* into the past while still presenting old data as live. We gave every viewer its own bounded queue instead.

Lost samples were corrupting the heart rate

When samples go missing the waveform is spliced together, and the interval across the join measures short. We traced one 0.4 s gap at 120 BPM turning a 500 ms interval into 100 ms. The detector now forgets its last beat across a gap, so it reports *no* interval rather than a wrong one.

Our first fix for that made things worse

We blanked detection for a long window after each gap, which seemed obviously right. Measuring it showed the opposite: it was throwing away real beats to prevent false ones that were not actually happening. We cut the window to 120 ms.

One user's sensor was stopping everyone else

Connecting a board switched the *shared* source, so a single viewer plugging in hardware ended every other viewer's trace. We now detach that viewer into a private session instead, which is what a sensor reading somebody's own chest should have been from the start.

Where we stand

COMPLETE AND VERIFIED

- ✓ Signal chain: filtering, detection, simulation
- ✓ 3-D anatomy, cardiac cycle, heart sounds
- ✓ Always-on deployment with real TLS
- ✓ Offline operation and installable app
- ✓ Browser DSP proven identical to the server's
- ✓ Shared, private and offline session models
- ✓ Automatic updates for web and installed copies
- ✓ CSV export, latency metering, continuous integration

OUTSTANDING

- Validation against a physical AD8232 board
- Electrode placement and skin-prep procedure
- Comparison against a reference monitor

Our honest position

We have run every path the hardware will take for hours against a modelled signal carrying the same gain, offset, quantisation, mains hum, motion artifacts and electrode dropout the real board will produce. **What remains is confirmation rather than construction.**

Bringing the hardware in

We modelled the AD8232 front end explicitly rather than generating a tidy waveform: $\times 1100$ gain, 1.5 V offset, 10-bit quantisation, Gaussian noise, mains hum and the transients an electrode makes when tugged. Because of that, a real trace misbehaving will point us at the physics rather than at the code.

STEP	WHAT IT ESTABLISHES	SUCCESS CRITERION
Bench bring-up	Board enumerates, sketch streams at 1000 Hz	Measured rate within 1% of 1000 Hz
Signal capture	Real PQRST morphology through the filter chain	P, QRS and T identifiable by eye
Rate agreement	Detector against a reference monitor	Within ± 2 BPM over 5 minutes
Artifact response	Leads-off and motion handling	Flagged, not silently mis-read
Latency	Electrode to animation, end to end	Under 150 ms

Past validation we intend to add multi-lead acquisition, arrhythmia classification, and stored sessions that can be reviewed afterwards. Moving the site onto a registered domain for production belongs to the same phase.

A working instrument, waiting on its sensor

1000 Hz

SAMPLE RATE

<150 ms

LATENCY BUDGET

16k

TRIANGLES, 1 DRAW CALL

0.0_{e+00}

DSP DIVERGENCE

We set out to make a heartbeat legible. What we have today is a measured signal chain, an anatomically faithful renderer, and a deployment that keeps working when the network does not. The address it answers on is temporary and moves to a registered domain for production.

Live system (temporary address)<https://143-198-27-18.nip.io>**Source**github.com/holarxmanuel/ecg-heart-visualizer

Not a medical device. For education and demonstration only.